

FAST · arXiv 2501.09747 · Physical Intelligence

Scientific Reviewer

8+3 · seat 4

Discrete tokenizer unlock / π_0 -FAST
Not FM retirement

Oct 19 paper club · VLA Architectures

MYTH BUST: FAST $\neq$ FM · $\pi_{0.5}$ = FAST pretrain $\rightarrow$ FM deploy

$\pi_0 \approx$ FM · π_0 -FAST $\approx$ discrete AR · $\pi_{0.5} \approx$ hybrid successor

CRITICAL MYTH BUST · say this first

FAST does NOT replace flow matching at deploy

Label	Meaning
FAST	Discrete DCT→BPE action tokenizer for AR VLAs
FM	Continuous flow-matching (π_0 / $\pi_{0.5}$ deploy)
π_0 -FAST	Discrete AR variant using this tokenizer
$\pi_{0.5}$	FAST for pretrain · FM for post-train/deploy

Naming: $\pi_0 \approx \text{FM}$ · $\pi_0\text{-FAST} \approx \text{discrete AR}$ · $\pi_{0.5} \approx \text{hybrid successor}$

Reviewer strengths

Causal story

Marginal information → didactic Fig. 3 links tokenization to AR failure

Simple compressor

Analytical DCT+BPE vs brittle VQ; few hypers; invertible

Broad empirics

Table I · FSQ · OpenVLA transfer · single-task · generalist · compute-matched

Honest latency

Concrete ms: ~100 FM vs ~750 AR-FAST on 4090

Open artifact

HF physical-intelligence/fast raises reproducibility

Reviewer weaknesses / threats

Figure-only scores

Hard to cite exact % without digitizing plots

FAST+ mix leakage

Train mix includes eval robots; Fig. 8 is cleaner OOD

Not FLOP-matched

Full LM AR vs small FM expert — different compute graphs

DROID n=44

Campus tour qualitative; modest quantitative suite

Preprint status

Venue / acceptance TENTATIVE on this PDF

Verdict

Credible, high-leverage systems + representation paper.

Cite as the discrete action tokenizer that unlocked
high-Hz AR VLAs and π_0 -FAST.

Cite $\pi_{0.5}$ separately for hybrid FAST $\rightarrow$ FM.

Do NOT cite FAST as having retired flow matching.

Not FM retirement · discrete tokenizer unlock / π_0 -FAST

approach that allows us to train the first autoregressive VLAs on dexterous and high-frequency robot data.

Action representations for VLA training. Prior works have explored various action parameterizations for training robot policies, including VLAs. One line of work uses “semantic” action representations like language sub-tasks [21, 2, 4], or keypoints [50, 32, 25, 19]. Such approaches can often learn from few examples or even perform tasks *zero-shot* without any robot examples [50, 32, 25], but require hand-designed low-level controllers for task execution, limiting their generality. An alternative approach directly trains VLAs to output low-level robot control commands given image and language instruction inputs. The most common design directly embeds actions into discrete tokens, that can be generated with standard autoregressive sequence models, like any popular vision-language model. Existing approaches map from continuous robot actions to discrete action tokens using a simple per-dimension, per-timestep binning scheme [9, 10, 39]. We find that this scheme struggles to scale to high-frequency robot control tasks. We propose a new tokenization scheme for robot actions, based on time-series compression techniques, that allows us to train autoregressive VLAs on high-frequency data. A number of works have also proposed alternatives to tokenization, for example by using regression heads or introducing new weights for diffusion decoding [20, 7, 41, 63]. In comparison, our approach does not require modifications of the underlying pre-trained transformer model, can easily be applied to any pre-trained autoregressive transformer model, and achieves competitive performance to state-of-the-art diffusion-based VLAs [7] across many tasks, while being significantly more compute efficient to train.

Another set of related work explores *vector-quantized* action representations [41, 3, 49]. Such approaches train a vector-quantized encoder-decoder network, for which reconstruction quality can be sensitive to hyperparameter choices and structure [66]. We find that these methods perform well at coarse, low-fidelity reconstruction tasks, but fail on high-frequency tasks when fine-grained control is required. In comparison, our FAST tokenization scheme has few hyperparameters and can reconstruct actions with high precision while offering strong compression properties.

III. PRELIMINARIES

Problem formulation. Our goal is to train policies $\pi(a_{1:H}|o)$ that map an observation o to a sequence of future robot actions $a_{1:H}$. We assume that policies output an “action chunk” [69, 40], a *sequence* of H actions [15, 7, 69], which makes it easier to produce temporally-consistent actions and reduces compounding error. The goal of *action tokenization* is to define a mapping $\mathcal{T}_a : a_{1:H} \rightarrow [T_1, \dots, T_n]$ from a sequence of continuous actions $a_{1:H}$, with dimensionality $|A|$, to a sequence of n discrete tokens $T \in |\mathcal{V}|$ from a vocabulary of size $|\mathcal{V}|$. Note that the number of tokens n may differ between action sequences, just like sentences of the same length may be tokenized into a variable number of text tokens.

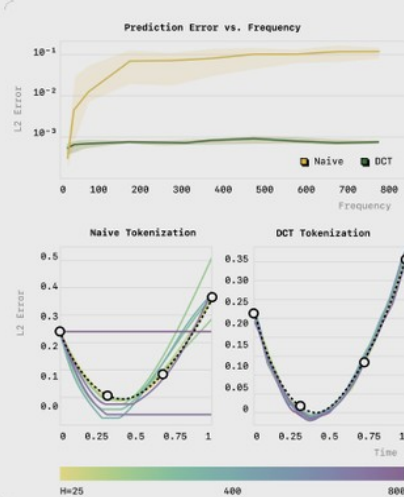


Fig. 3: **Effect of sampling rate on prediction performance.** We train a small autoregressive transformer model on a didactic interpolation task, in which the network must predict the black dashed curve given the four circles. We find that models trained with the binning tokenization approach used in prior VLAs [10, 39] produce increasingly poor predictions as we increase the sampling frequency of the underlying signal, due to strong correlation between consecutive tokens at high frequencies. Our FAST tokenization approach, based on the discrete cosine transform (DCT), addresses the problem and leads to high-quality predictions across all sampling rates.

Binning-based action tokenization. The most commonly used approach for action tokenization is a simple binning discretization scheme [8, 10, 39, 72, 56]. For a given action a , this approach discretizes each dimension independently, dividing the range of values in the training dataset into N uniform bins, most commonly using $N = 256$. For a *sequence* of D -dimensional actions $a_{1:H}$, this tokenization scheme would be applied to each time step, resulting in a final token sequence $\mathcal{T}_a(a_{1:H}) = [T_{1,1}, \dots, T_{1,D}, \dots, T_{H,1}, \dots, T_{H,D}]$. For high-frequency robot data, this tokenization scheme is sub-optimal: it can easily produce hundreds of tokens per action chunk, which make training challenging and lead to slow inference.

IV. CASE STUDY: HOW DOES TOKENIZATION AFFECT VLA TRAINING?

To illustrate the challenge of training autoregressive policies with current action tokenization approaches, we start

Fig.9 FAST vs diffusion π_0

which the tokenizer is applied to a completely new robot setup without recomputing the tokenization. We find that the FAST+ tokenizer achieves good compression performance across a wide range of robot datasets, reducing the number of action tokens by 2x across all datasets, and significantly more on some.

We also test performance of the universal tokenizer for policy training, and report results alongside the per-dataset tokenizers in Figure 6. Across all tasks, the *universal* tokenizer closely matches the performance of the dataset-specific FAST tokenizers, suggesting that the universal tokenizer can be used as a strong default for robot action tokenization.

D. Ablation Studies

We analyze two key aspects of our method: (1) Is our FAST tokenization approach *independent* of the underlying VLA backbone? (2) How important is the BPE compression step, the only learned component of our tokenization pipeline.

To answer the first question, we train an OpenVLA policy [39] on the challenging high-frequency T-shirt folding dataset, comparing the naïve tokenization approach originally used in OpenVLA to our FAST+ tokenizer. To comply with the task setup, we modify the OpenVLA model code to accept multiple input images and predict 1-second action chunks. The results on the right demonstrate that FAST is able to significantly boost performance of OpenVLA, enabling it to train effectively on high-frequency robot manipulation data. This suggests, that our tokenization approach is *independent* of the underlying model backbone, and may be easily applied to a wide range of pre-trained autoregressive transformer models.

Secondly, we ablate the BPE encoding step on the table bussing and T-shirt folding tasks. The figure on the right shows that the resulting policies *without BPE encoding* achieve worse rollout performance (but still outperform naïve tokenization). Intuitively, the DCT transform still concentrates most of the signal's information in a few tokens, improving the learning signal. However, without BPE, there is a large number of repeated 0-tokens which dilute the learning signal and also significantly slow down inference, since models need to autoregressively predict hundreds of action tokens, ultimately leading to worse policy performance.

E. Comparing FAST to Diffusion

In this section, we compare π_0 , a state-of-the-art diffusion VLA, to our model that combines π_0 with FAST and uses

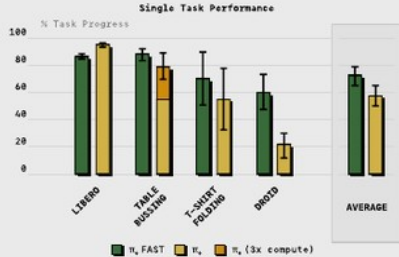


Fig. 9: Comparison of diffusion π_0 [7] to our π_0 model with FAST decoding on single-task training. On small datasets (Libero, T-Shirt Folding), both perform comparably. On large datasets (Table Bussing), FAST converges faster. In DROID, we find that FAST follows language instructions better. We report mean and 95% CI.

autoregressive decoding. We compare the performance of both models on the tasks from Section VI-B.

We report results in Figure 9. We find that on small datasets (Libero, T-Shirt Folding; <50h), both VLAs perform comparably. However, on large datasets like Table Bussing, we find that the FAST-based VLA converges significantly faster, reaching high performance with 3x fewer training steps than the diffusion variant of π_0 . Additionally, we find that the autoregressive π_0 model trained with FAST tokenization follows language instructions more closely: in the DROID evaluations, the diffusion π_0 model often ignores the language instructions, leading to a lower score. We will leave a detailed investigation of the language following abilities of diffusion and autoregressive VLAs to future work.

One current limitation of the autoregressive VLA is its inference speed: while π_0 with diffusion typically predicts one second action chunks within 100ms on an NVIDIA 4090 GPU, the π_0 model with FAST tokenization needs approximately 750ms of inference time per chunk, since it must perform more autoregressive decoding steps (typically 30-60 action tokens need to be decoded, vs. 10 diffusion steps for diffusion π_0) and use the full 2B parameter language model backbone for autoregressive decoding (vs. a 300M parameter “action expert” for diffusion π_0). While we did not find this slower inference to hurt performance on the static manipulation tasks we evaluated, it made evaluations significantly slower. Going forward, there are many techniques for accelerating the inference of discrete, autoregressive transformer models that are used extensively in the LLM literature (e.g., speculative decoding, quantization, custom inference kernels, etc.), but we will leave an investigation of these to future work.

Inference latency · RTX 4090 class

~100 ms FM vs ~750 ms AR-FAST per 1 s chunk (~7.5×)

π_0 FM / diffusion

~100 ms

~10 FM steps × ~300M expert

π_0 -FAST AR

~750 ms

~30-60 AR tokens × full ~2-3B LM

Static manip OK · dynamic closed-loop / eval throughput hurt

Future: speculative decode · quant · kernels · Empiricist must log ms/chunk

Naming hygiene tree · Archaeologist / club spine

Never collapse these three labels

π_0

≈ continuous FM
PaliGemma + action expert
~100 ms / chunk deploy

π_0 -FAST

≈ discrete AR sibling
Same backbone + FAST tokens
~750 ms / chunk · 5× train

$\pi_{0.5}$

≈ hybrid successor
FAST pretrain → FM post
FM RETAINED at deploy

Forward lineage (CLEAR)

FAST

π_0 -FAST

$\pi_{0.5}$
(FAST→FM)

KI

RTC

BEHAVIOR

Scientific Reviewer · close

Cite as discrete tokenizer unlock / π_0 -FAST — not FM retirement.

Caveats: figure-only scores · FAST+ mix leakage · AR vs FM not FLOP-matched · DROID n=44

Q&A · Oct 19 · FAST / 2501.09747

FAST: Efficient Action Tokenization for Vision-Language-Action Models

Karl Pertsch^{*,1,2,3}, Kyle Stachowicz^{*,2},
Brian Ichter¹, Danny Driess¹, Suraj Nair¹, Quan Vuong¹, Oier Mees², Chelsea Finn^{1,3}, Sergey Levine^{1,2}

¹Physical Intelligence, ²UC Berkeley, ³Stanford

<https://pi.website/research/fast>

Abstract—Autoregressive sequence models, such as Transformer-based vision-language action (VLA) policies, can be tremendously effective for capturing complex and generalizable robotic behaviors. However, such models require us to choose a tokenization of our continuous action signals, which determines how the discrete symbols predicted by the model map to continuous robot actions. We find that current approaches for robot action tokenization, based on simple per-dimension, per-timestep binning schemes, typically perform poorly when learning dexterous skills from high-frequency robot data. To address this challenge, we propose a new compression-based tokenization scheme for robot actions, based on the discrete cosine transform. Our tokenization approach, Frequency-space Action Sequence Tokenization (FAST), enables us to train autoregressive VLAs for highly dexterous and high-frequency tasks where standard discretization methods fail completely. Based on FAST, we release FAST+, a *universal* robot action tokenizer, trained on 1M real robot action trajectories. It can be used as a black-box tokenizer for a wide range of robot action sequences, with diverse action spaces and control frequencies. Finally, we show that, when combined with the π_0 VLA, our method can scale to training on 10k hours of robot data and match the performance of diffusion VLAs, while reducing training time by up to 5x.

I. INTRODUCTION

Large, high-capacity Transformer models can be tremendously effective for capturing complex and generalizable robotic behaviors both from scratch [8, 69, 51, 6, 20, 62] and using models pre-trained for next-token prediction on Internet-scale image-text corpora [10, 39, 63, 7, 65]. However, these models require choosing a tokenization of the continuous action signal, which determines how the discrete symbols predicted by the model map to continuous robot actions [64, 34, 41, 12]. It is widely known that a good choice of tokenization can be critical to the performance of sequence models [55, 57]. Prior robotic policies of this sort typically use naïve tokenization strategies based on a per-dimension, per-timestep binning scheme [9, 10, 39]. We find that such methods perform poorly when learning dexterous skills with high-frequency control (see Figure 2, right). We observe that correlations between time steps are a major challenge for naïve tokenization strategies when predicting sequences of

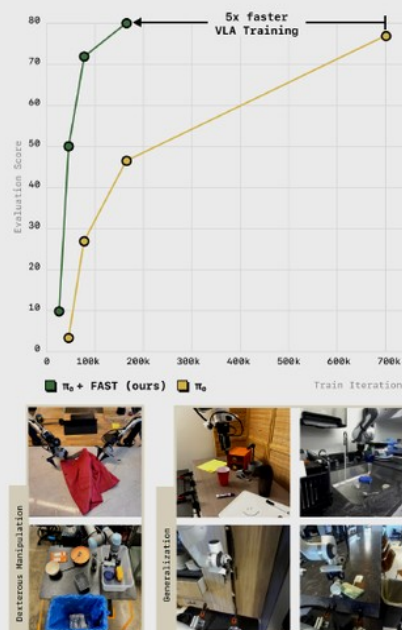


Fig. 1: We propose FAST, a simple yet effective approach for tokenization of robot action trajectories via time-series compression. FAST enables training of autoregressive VLAs that solve complex dexterous manipulation tasks and generalize broadly to new scenes. We use it to train π_0 -FAST, a generalist robot policy that matches the performance of the state-of-the-art π_0 diffusion VLA on dexterous and long-horizon manipulation tasks, while training 5x faster (top).

*: Core contributors
Correspondence to: research@physicalintelligence.company

arXiv:2501.09747v1 [cs.RO] 16 Jan 2025